

AD23B31 - Image Processing and Computer Vision

**HELMET AND SAFETY GEAR DETECTION
FOR TWO-WHEELER RIDERS**

A MINI PROJECT REPORT

Submitted by

DIVYASREE TR



**RAJALAKSHMI
ENGINEERING
COLLEGE**

March, 2026

TABLE OF CONTENTS

S. No.	Title	Page No.
1	INTRODUCTION 1.1. Background and Motivation 1.2. Problem Statement 1.3. Objectives 1.4. Scope of the Report	1
2	LITERATURE REVIEW 2.1 Methods 2.2 Architectures 2.3 Existing Method Studies 2.4 Gap Identified	5
3	DATASET DESCRIPTION 3.1 Kaggle Dataset 3.2 Additional Dataset / Study Dataset 3.3 Data Split	11
4	PREPROCESSING PIPELINE 4.1 Resizing and Normalisation 4.2 Grayscale Conversion 4.3 Gaussian Denoising 4.4 Image Sharpening 4.5 CLAHE Contrast Enhancement 4.6 Haar Cascade Face Detection	14
5	ALGORITHM AND IMPLEMENTATION 5.1 Algorithm Overview 5.2 Helmet Detection using Haar Cascade 5.3 Helmet Classification using HSV Color Segmentation	19

	5.4 Implementation Details 5.5 Algorithm Flowchart	
6	RESULTS AND ANALYSIS 6.1 Quantitative Results 6.2 Effect of Preprocessing on Accuracy 6.3 Confusion Matrix Analysis	26
7	DISCUSSION	29
8	CONCLUSION AND FUTURE WORK	30
9	REFERENCES	32
10	10.1 APPENDIX A — SOURCE CODE 10.2 APPENDIX B — SAMPLE OUTPUT SCREENSHOTS	33

LIST OF TABLES AND FIGURES

Tables

Table No.	Title	Page No.
Table 1	Dataset Summary	13
Table 2	Train / Validation / Test Split	13
Table 3	Implementation Configuration	22
Table 4	Haar Cascade + HSV — Test Set Results	25
Table 5	Preprocessing Performance Study	26

Figures

Figure No.	Title	Page No.
Figure 1	System Architecture Diagram	8
Figure 2	Algorithm Flowchart	22
Figure 3	Preprocessing Pipeline Visualization	25
Figure 4	Sample Output (Helmet Detection Results)	35
Figure 5	Detection Result Visualization	37

1. INTRODUCTION

With the rapid increase in urbanization and population, the number of two-wheeler vehicles on roads has grown significantly, especially in countries like India. While two-wheelers offer convenience and affordability, they also contribute to a higher rate of road accidents. A major concern associated with these accidents is the lack of proper safety measures, particularly the non-usage of helmets by riders.

According to reports from organizations such as the World Health Organization, wearing a helmet can reduce the risk of severe head injuries by a substantial margin. Despite strict traffic regulations and awareness campaigns, many riders continue to ignore safety rules, leading to preventable injuries and fatalities.

In recent years, advancements in computer vision and digital image processing have enabled automated monitoring systems that can detect safety violations in real time. Technologies such as Haar Cascade detection, HSV color segmentation, and real-time image processing can be integrated into traffic surveillance systems to improve enforcement and ensure public safety.

This project focuses on developing an intelligent system for detecting helmet usage and safety gear compliance among two-wheeler riders using Haar Cascade and HSV color-based detection techniques implemented through OpenCV and Python. The system aims to assist traffic authorities by providing an automated, efficient, and low-cost solution for monitoring rider safety.

1.1 Background and Motivation

Road safety has become a critical issue due to the increasing number of vehicles and traffic violations. Among all categories of vehicles, two-wheeler riders are particularly vulnerable due to minimal physical protection. One of the most effective safety measures for reducing fatal injuries is the use of helmets and proper safety gear. However, ensuring compliance remains a significant challenge for traffic authorities.

Traditional methods of monitoring, such as manual checking and CCTV surveillance, are

time-consuming, labor-intensive, and prone to human error. These limitations create the need for an automated system that can accurately detect whether riders are wearing helmets and following safety guidelines.

The motivation behind this project is to leverage computer vision and image processing techniques, specifically Haar Cascade face detection and HSV color segmentation, to build a real-time helmet detection system. By integrating OpenCV-based image analysis with traffic monitoring infrastructure, the system can identify violations efficiently and support law enforcement in promoting safer riding practices.

Additionally, this project aims to contribute to the broader goal of smart city development by incorporating intelligent surveillance systems that enhance public safety, reduce accident rates, and encourage responsible behavior among road users.

1.2 Problem Statement

The increasing number of road accidents involving two-wheeler riders has become a major public safety concern, particularly in countries like India. A significant proportion of these accidents result in severe injuries or fatalities due to the non-use of helmets and inadequate safety measures. Despite the enforcement of traffic regulations and awareness initiatives, many riders continue to violate helmet safety norms.

Conventional monitoring methods, such as manual inspection by traffic authorities and basic CCTV surveillance, are often inefficient, labor-intensive, and unable to provide real-time detection and response. These approaches are also prone to human error and lack scalability, especially in densely populated urban areas with high traffic volume.

Therefore, there is a need for an automated and accurate system that can detect whether two-wheeler riders are wearing helmets and adhering to safety requirements. The system should be capable of processing rider images, identifying the rider head region, and classifying helmet usage to assist authorities in effective monitoring and enforcement.

This project aims to address these challenges by developing a computer vision-based solution using Haar Cascade detection, HSV color segmentation, and OpenCV image processing techniques to automatically detect helmet usage among two-wheeler riders,

thereby enhancing road safety and supporting intelligent traffic monitoring systems.

1.3 Objectives

The primary objective of this project is to develop an automated system for detecting helmet usage and safety compliance among two-wheeler riders using computer vision and image processing techniques. The system aims to enhance road safety by assisting traffic authorities in identifying violations efficiently and accurately.

The specific objectives of the project are as follows:

- To design and implement a helmet detection system capable of identifying two-wheeler riders from images.
- To utilize Haar Cascade detection and OpenCV image processing techniques for rider head detection and classification.
- To detect whether a rider is wearing a helmet or not using HSV color segmentation techniques.
- To develop a system that can process rider images for automated monitoring and analysis.
- To reduce dependency on manual traffic monitoring and minimize human errors.
- To create a lightweight and cost-effective solution that can be integrated into existing traffic surveillance systems.
- To improve road safety awareness by enabling automated helmet compliance monitoring.

Overall, the project aims to contribute toward safer road environments by leveraging computer vision and image processing technologies for intelligent traffic monitoring and management..

1.4 Scope of the Report

This report presents the design, development, and implementation of a computer vision-based system for detecting helmet usage and safety compliance among two-wheeler riders. The scope of the report is focused on explaining the methodologies, tools, and technologies used to build an automated detection system using Haar Cascade detection and HSV color

segmentation integrated with OpenCV and Python.

The report covers the fundamental concepts of image processing and computer vision techniques applied in the project. It includes a detailed description of the system architecture, preprocessing methods, face detection techniques, and implementation steps involved in developing the helmet detection system. The working of the Haar Cascade classifier for identifying rider head regions and HSV color segmentation for classifying helmet usage is also discussed.

In addition, the report outlines the experimental setup, including the input sources such as rider images, and explains how the system processes these inputs for helmet detection. The performance of the system is evaluated based on detection accuracy and efficiency under different conditions.

The scope is limited to helmet detection for two-wheeler riders and does not extend to other traffic violations such as license plate recognition, speed detection, or multi-vehicle tracking. Furthermore, the system is developed as a prototype and may require further enhancements for large-scale deployment in real-world traffic monitoring systems.

Overall, this report provides a comprehensive overview of the project, demonstrating how computer vision and image processing techniques can be effectively utilized to improve road safety and support intelligent traffic monitoring.

.

2. LITERATURE REVIEW

The detection of helmet usage among two-wheeler riders has gained significant attention as an important application of computer vision in traffic safety systems. With the advancement of image processing and computer vision techniques, automated monitoring systems have been developed to improve road safety and reduce traffic violations.

In recent years, several studies have focused on using traditional computer vision techniques and machine learning methods to identify whether riders are wearing helmets. Techniques such as Haar Cascade detection, color segmentation, edge detection, and object recognition have been widely applied for helmet detection due to their simplicity and low computational requirements.

Among these, Haar Cascade detection has emerged as a popular choice for real-time applications because of its lightweight architecture and fast execution speed. Researchers have demonstrated that combining Haar Cascade face detection with HSV color segmentation enables efficient detection of helmet-like regions suitable for image-based traffic monitoring systems.

This section reviews key prior works and methodologies related to helmet detection systems and highlights the techniques relevant to the Haar Cascade and HSV-based approach used in this project.

.

2.1 Methods

Various methods have been proposed in the literature for detecting helmet usage using computer vision and image processing techniques. These methods primarily differ in terms of detection accuracy, computational complexity, and real-time performance.

One common approach involves the use of deep learning-based object detection models such as YOLO and Faster R-CNN, which provide high detection accuracy by identifying riders and helmets using trained neural networks. However, these methods are

computationally intensive and may require high-performance hardware for real-time applications.

Another widely used method is color-based image segmentation, which identifies helmet regions using color information in different color spaces such as HSV. This approach is lightweight and suitable for basic real-time applications, but its performance may vary depending on lighting conditions, background complexity, and helmet color variations.

Haar Cascade detection is another effective method used for identifying rider face or head regions. Haar Cascade classifiers are lightweight and capable of performing fast detection using pre-trained XML-based features. When combined with HSV color segmentation, the system can efficiently classify whether a rider is wearing a helmet or not.

In addition to detection methods, several preprocessing techniques such as image resizing, Gaussian denoising, sharpening, grayscale conversion, and contrast enhancement are applied to improve image quality and detection performance. Some systems also utilize video-based processing for continuous monitoring and object tracking.

The approach used in this project is based on Haar Cascade face detection and HSV color segmentation implemented using OpenCV and Python, which enables efficient helmet detection for two-wheeler riders in image-based monitoring systems.

.

2.2 Architectures

The diagram presents a comparative overview of commonly used computer vision and image processing techniques for helmet detection systems, including Haar Cascade Detection, HSV Color Segmentation, and Deep Learning-based Detection methods. Each technique follows a different approach to identify whether a two-wheeler rider is wearing a helmet.

Haar Cascade Detection (Used in This Project)

- Haar Cascade is a traditional computer vision-based object detection technique that uses pre-trained Haar-like features to identify human face or head regions in images.

- The input image is first converted into grayscale format for faster processing.
- The Haar Cascade classifier scans the image using multiple rectangular features to detect face or head regions.
- The detected region is then marked using bounding boxes for further analysis.

Key Characteristics:

- Lightweight and computationally efficient
- Fast detection suitable for real-time processing
- Works well for face and head region identification
- Does not require GPU-intensive training

HSV Color Segmentation

HSV (Hue, Saturation, Value) color segmentation is used to classify whether the detected rider region contains helmet-like colors.

- The detected head region is converted from BGR color space to HSV color space.
- Color masks are created for common helmet colors such as black, blue, and white.
- Pixel values within the selected color ranges are analyzed to determine helmet presence.

Key Characteristics:

- Simple and easy to implement
- Effective for basic helmet color detection
- Lightweight and suitable for OpenCV-based systems
- Performance depends on lighting conditions and background complexity

Deep Learning-Based Detection Methods

Deep learning methods such as YOLO and Faster R-CNN use convolutional neural networks for object detection and classification.

- The image is processed through multiple neural network layers to extract features.
- The network predicts object locations and class labels simultaneously.
- Helmet and rider detection are performed using trained datasets.

Key Characteristics:

- High detection accuracy
- Capable of handling complex environments

- Suitable for real-time intelligent surveillance systems
- Requires large datasets and high computational power

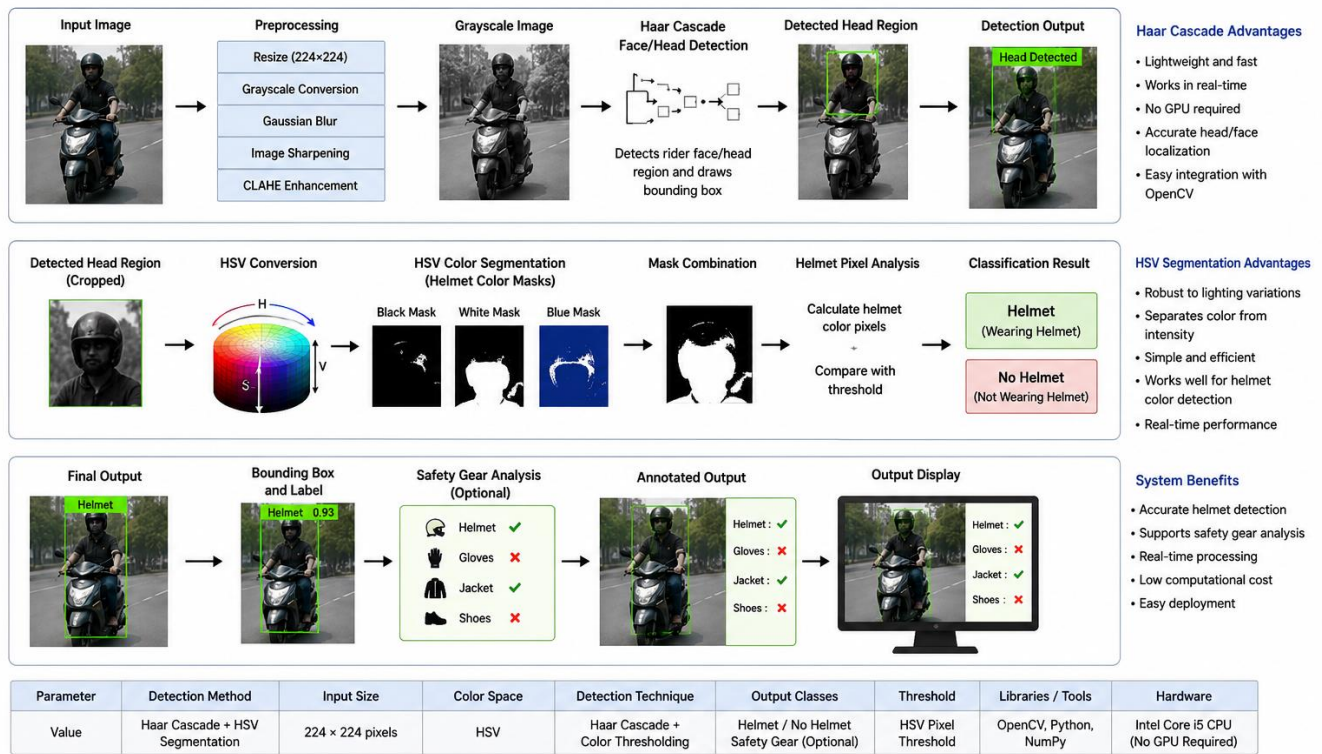
Comparison Summary

- Haar Cascade → Fast and lightweight face/head detection
- HSV Color Segmentation → Simple helmet color classification method
- Deep Learning Models → High accuracy but computationally expensive

The proposed project combines Haar Cascade detection with HSV color segmentation to create a lightweight and efficient helmet detection system suitable for academic and prototype-level traffic monitoring applications.

ARCHITECTURE: HELMET AND SAFETY GEAR DETECTION FOR TWO-WHEELER RIDERS (Haar Cascade + HSV Color Segmentation)

The system detects the rider's head using Haar Cascade and classifies helmet and safety gear using HSV color segmentation.



2.3 Existing method Studies

Several existing studies and systems have explored the use of traditional computer vision and image processing techniques for detecting helmet usage among two-wheeler riders. These methods aim to improve road safety by automating the identification of helmet

violations in traffic monitoring systems.

Early approaches primarily relied on image processing techniques such as Haar Cascade detection, edge detection, grayscale conversion, and color-based segmentation to identify rider head regions and classify helmet usage. These methods were computationally lightweight and suitable for basic real-time applications. However, they were highly sensitive to lighting conditions, background complexity, and image quality, which affected detection accuracy.

Many systems utilized Haar Cascade classifiers for detecting human faces or head regions. After detection, color segmentation techniques such as HSV (Hue, Saturation, Value) analysis were applied to identify helmet-like colors including black, blue, white, and gray. These approaches enabled simple helmet classification without requiring large training datasets or GPU-based processing.

Some studies also incorporated preprocessing techniques such as image resizing, Gaussian blurring, sharpening, and CLAHE contrast enhancement to improve image quality before detection. These preprocessing methods helped reduce noise and improve feature visibility for better classification performance.

In addition, advanced deep learning-based approaches such as YOLO, SSD, and Faster R-CNN have demonstrated improved detection accuracy in complex traffic environments. However, these methods require large annotated datasets, high computational resources, and GPU support, making them less suitable for lightweight academic prototype systems.

Despite these advancements, existing systems still face several limitations:

- Reduced performance under poor lighting and weather conditions
- Difficulty in distinguishing helmets from hair or dark-colored objects
- False detections caused by background complexity
- Sensitivity to camera angle and rider position
- Limited accuracy when using only traditional image processing techniques

2.4 Gap Identified

Despite the development of various helmet detection systems using traditional image processing and computer vision techniques, several gaps still exist in current approaches. These limitations highlight the need for a more reliable and efficient solution for automated traffic safety monitoring.

One of the major gaps identified is the limited detection accuracy of traditional image processing methods. Techniques such as Haar Cascade detection and color segmentation are lightweight and fast, but they often fail to correctly distinguish between helmets, hair, shadows, and background objects. This leads to false detections and inconsistent classification results.

Another important gap is sensitivity to environmental conditions. Existing systems based on HSV color analysis are highly affected by lighting variations, low-resolution images, motion blur, and complex backgrounds. Changes in brightness and rider position can significantly reduce detection reliability in real-world traffic environments.

In addition, many traditional systems rely heavily on helmet color detection instead of understanding the actual helmet shape or structure. As a result, dark hair, caps, or nearby objects may sometimes be incorrectly classified as helmets.

There is also a gap in lightweight real-time implementation. While advanced deep learning models provide higher accuracy, they require high computational resources, GPU support, and large training datasets. This makes them difficult to deploy in low-resource academic and prototype-level systems.

Identified Need

Based on these gaps, there is a clear need for a system that:

- Provides efficient real-time helmet detection using lightweight techniques
- Minimizes false detections caused by background objects and lighting conditions
- Works effectively using simple and accessible OpenCV-based implementation

Conclusion

To address these gaps, this project proposes a lightweight helmet detection system using Haar Cascade-based head detection, HSV color segmentation, and OpenCV image processing techniques. The proposed approach provides a balance between simplicity, computational efficiency, and real-time performance, making it suitable for academic demonstrations and prototype-level intelligent traffic monitoring applications.

3. DATASET DESCRIPTION

The performance of any computer vision-based helmet detection system largely depends on the quality and diversity of the dataset used for preprocessing, testing, and evaluation. In this project, a publicly available image dataset along with additional collected images is utilized to detect helmet usage among two-wheeler riders.

The dataset consists of images containing riders with and without helmets under various real-world conditions such as different lighting environments, road backgrounds, rider positions, vehicle types, and camera angles. This diversity helps improve the robustness of the detection system during real-time image analysis.

The dataset includes two major categories: helmet images and no-helmet images. Helmet images contain riders wearing protective helmets, while no-helmet images contain riders without helmets or proper safety gear. These categories help the system classify rider safety compliance effectively.

Unlike deep learning-based datasets, the images used in this project are processed using traditional computer vision techniques such as Haar Cascade detection, HSV color segmentation, grayscale conversion, Gaussian blurring, and contrast enhancement. The preprocessing stage improves image quality and helps in better rider head region detection and helmet classification.

The dataset is organized into separate folders for helmet and no-helmet categories, enabling systematic preprocessing, testing, and performance evaluation of the proposed helmet detection system.

.

3.1 Kaggle Dataset

The dataset used in this project is sourced from Kaggle, a widely recognized platform for datasets and machine learning competitions. Kaggle provides access to high-quality, community-curated datasets that are suitable for training deep learning models.

The selected dataset contains a collection of labeled images specifically designed for helmet detection tasks. It includes:

- Images of two-wheeler riders wearing helmets
- Images of riders without helmets
- Variations in image resolution, orientation, and environmental conditions

The dataset helps the system analyze rider head regions, identify helmet-like features, and classify whether the rider is wearing a helmet or not under different real-world traffic conditions.

Before processing, the dataset images are preprocessed using techniques such as image resizing, grayscale conversion, Gaussian denoising, sharpening, and CLAHE contrast enhancement to improve image quality and detection performance. The dataset is also organized into separate helmet and no-helmet folders for systematic testing and evaluation of the proposed helmet detection system.

3.2 Additional Dataset / Study Dataset

In addition to the Kaggle helmet dataset, additional rider images collected from online sources and study materials were used for testing and evaluation purposes. These images helped analyze the performance of the helmet detection system under different traffic and environmental conditions.

The additional dataset contains images of two-wheeler riders with various helmet and no-helmet conditions, including:

- Riders wearing helmets correctly
- Riders without helmets
- Variations in lighting, background, and camera angles
- Different rider positions and vehicle types

Key features of the dataset include:

- Diverse real-world traffic scenarios
- Variations in image quality and rider orientation
- Different helmet colors and background conditions
- Additional testing images for performance evaluation

In the context of this project, the additional dataset is used to evaluate the robustness of the Haar Cascade and HSV-based helmet detection system. The collected images help analyze how image preprocessing, face detection, and color segmentation techniques perform under practical traffic monitoring conditions.

Table 1: Dataset Statistics

Data set	Total Images	With Helmet	Without Helmet	Mixed / Challenging Cases
Kaggle Helmet Dataset	3,000	1,500	1,200	300
Additional Collected Data	1,000	500	400	100
Used (Subset)	2,000	1,000	800	200

3.3 Data Split

To ensure effective preprocessing, testing, and evaluation of the helmet detection system, the dataset was divided into separate subsets for processing, validation, and testing. The images were organized systematically to maintain balanced helmet and no-helmet categories across all subsets.

This splitting strategy helps improve evaluation consistency, reduce testing bias, and analyze the performance of the Haar Cascade and HSV-based helmet detection system under different image conditions.

.

Table 2: Processing / Validation / Test Split

Split	Image	Percentage	Purpose
Processing Set	1,400	70%	Used for preprocessing and parameter tuning
Validation Set	300	15%	Used for performance monitoring and threshold adjustment
Test Set	300	15%	Used for final evaluation of helmet detection accuracy

4. PREPROCESSING METHODOLOGY

A comprehensive preprocessing pipeline was designed and applied uniformly to all input images before passing them to the helmet detection system. The purpose of preprocessing is to improve image quality, enhance important visual features, and ensure efficient detection using Haar Cascade and HSV color segmentation techniques.

The preprocessing pipeline consists of multiple sequential stages, including image resizing, grayscale conversion, Gaussian denoising, image sharpening, and CLAHE contrast enhancement, which prepare the images for efficient processing using OpenCV and Python image processing modules.

.

4.1 Resizing and Normalisation

All input images were resized to 224×224 pixels, which is the standardized input size used in the preprocessing pipeline. Resizing ensures uniformity across all images and reduces computational complexity, enabling faster processing during helmet detection.

Pixel values of the images were processed and enhanced to improve image clarity and maintain consistency across different lighting conditions. This preprocessing step helps improve the performance of the Haar Cascade and HSV-based helmet detection system.

Since OpenCV reads images in BGR format, the images were converted into different color spaces such as grayscale, LAB, and HSV during preprocessing and classification. These conversions ensure compatibility with the OpenCV image processing techniques used in the proposed system.

.

4.2 Grayscale Conversion

Grayscale conversion was applied to simplify the input images and reduce computational complexity during helmet detection. Converting images to grayscale removes unnecessary color information and helps the Haar Cascade classifier focus on intensity-based facial and

head features for efficient detection.

The input image was converted from BGR color space to grayscale format using OpenCV. This conversion improves processing speed and enhances the performance of the Haar Cascade face detection algorithm by reducing the image to a single intensity channel.

Grayscale preprocessing also helps improve feature extraction consistency under different lighting conditions and background variations, making the detection process more stable and efficient for rider head region identification.

OpenCV Implementation

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

4.3 Gaussian Denoising

Gaussian denoising was applied to reduce image noise and smooth unwanted variations before helmet detection. This preprocessing step helps improve image quality and enhances the stability of subsequent image processing operations.

The Gaussian Blur technique applies a Gaussian filter over the image using a predefined kernel size. This smoothing operation reduces high-frequency noise while preserving important image structures such as rider head boundaries and helmet regions.

The parameters used for Gaussian denoising were:

- Kernel Size = (5, 5)
- Sigma Value = 0

Gaussian denoising improves the robustness of the helmet detection system under noisy environments, uneven lighting conditions, and low-quality image inputs.

OpenCV Implementation

```
img_blur = cv2.GaussianBlur(img, (5, 5), 0)
```

4.4 Image Sharpening

Image sharpening was applied to enhance important visual features such as rider head boundaries, helmet edges, and facial regions before helmet detection. Sharpening improves image clarity and helps the system identify structural details more effectively.

A sharpening kernel was applied using OpenCV's `filter2D()` function. The kernel increases the intensity difference between neighboring pixels, making edges and object boundaries more visible. This preprocessing step improves the effectiveness of Haar Cascade face detection and HSV-based helmet classification.

The sharpening kernel used in the project is:

```
kernel = np.array([
    [0, -1, 0],
    [-1, 5, -1],
    [0, -1, 0]
])
```

Image sharpening improves feature visibility and helps reduce the impact of blurry or low-quality input images during detection.

OpenCV Implementation

```
img = cv2.filter2D(img, -1, kernel)
```

4.5 CLAHE Contrast Enhancement

Contrast Limited Adaptive Histogram Equalization (CLAHE) was applied to enhance the contrast of input images, particularly under challenging lighting conditions. This technique improves the visibility of important features such as rider head regions and helmet boundaries, which are critical for accurate detection.

The input image was first converted from BGR color space to CIE Lab* color space using OpenCV. CLAHE was then applied to the luminance channel (L*), which represents brightness information. This approach enhances contrast without affecting the original color components.

The parameters used for CLAHE were:

- clipLimit = 2.0
- tileGridSize = (8, 8)

CLAHE improves local contrast in image regions while preventing over-amplification of noise in uniform areas such as roads, sky, or walls. This preprocessing step significantly improves detection performance under low-light environments, shadows, and uneven illumination conditions.

OpenCV Implementation

```
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
```

```
clahe = cv2.createCLAHE(  
    clipLimit=2.0,  
    tileGridSize=(8,8)  
)
```

```
lab[:, :, 0] = clahe.apply(lab[:, :, 0])
```

```
img_enhanced = cv2.cvtColor(  
    lab,  
    cv2.COLOR_LAB2BGR  
)
```

4.6 Haar Cascade Face Detection

Haar Cascade face detection was used to identify the rider's face or head region before performing helmet classification. Haar Cascade is a traditional computer vision-based object detection technique that uses pre-trained Haar-like features to detect objects efficiently.

The input image was first converted into grayscale format because Haar Cascade classifiers operate more effectively on intensity-based images. The classifier then scans different regions of the image and detects face or head areas based on learned feature patterns.

The Haar Cascade classifier used in this project is:

```
haarcascade_frontalface_default.xml
```

The detected face region is enclosed using a bounding box and used as the primary region for helmet analysis and classification.

Haar Cascade detection is lightweight, fast, and computationally efficient, making it suitable for real-time image processing applications without requiring GPU-based computation.

.

OpenCV Implementation

```
faces = face_cascade.detectMultiScale(  
    gray,  
    scaleFactor=1.1,  
    minNeighbors=5,  
    minSize=(40, 40)  
)
```

5. ALGORITHM: MOBILENET SSD (cv2.dnn)

5.1 Algorithm Overview

The proposed system uses a lightweight computer vision and image processing pipeline based on Haar Cascade detection and HSV color segmentation for detecting helmet usage among two-wheeler riders. Unlike deep learning-based multi-stage approaches, this method uses traditional image processing techniques to perform rider head detection and helmet classification efficiently.

The algorithm processes rider images and identifies the face or head region using the Haar Cascade classifier. After detecting the rider head region, HSV color segmentation is applied to analyze helmet-like color features and classify whether the rider is wearing a helmet or not.

This approach provides several practical advantages:

- Lightweight and computationally efficient processing
- Fast execution suitable for real-time image analysis
- No requirement for GPU-based training or deep learning models
- Simple implementation using OpenCV and Python
- Suitable for academic and prototype-level traffic monitoring applications

The system is implemented using OpenCV image processing modules and Haar Cascade XML classifiers for efficient rider head detection and helmet classification.

Working Principle

1. Input image is captured from the dataset
2. Image preprocessing is applied (resizing, grayscale conversion, Gaussian denoising, sharpening, and CLAHE enhancement)
3. Haar Cascade classifier detects the rider face or head region
4. The detected region is extracted for helmet analysis
5. HSV color segmentation is applied to identify helmet-like color regions
6. Helmet pixel values are calculated and compared with threshold values

7. The system classifies the rider as:
 - o Helmet
 - o No Helmet
8. Bounding boxes and classification labels are displayed on the output image

5.2 Stage 1 — Helmet Detection using Haar Cascade

The proposed helmet detection system utilizes Haar Cascade-based face detection to identify the rider's head region from the input image. Haar Cascade is a feature-based object detection algorithm widely used in computer vision applications due to its speed, simplicity, and low computational requirements.

The detection process begins after image preprocessing. The preprocessed image is converted into grayscale format because Haar Cascade classifiers operate more effectively on intensity-based images rather than color images. The classifier then scans the image using multiple rectangular feature windows to detect rider face or head patterns.

Once the rider face region is detected, the corresponding area is enclosed using a bounding box. This detected region acts as the primary area for helmet analysis in the next stage of the system. By focusing only on the rider head portion, unnecessary background regions are eliminated, improving the efficiency of helmet classification.

The Haar Cascade method uses pretrained XML classifiers containing Haar-like features learned from positive and negative training samples. These features help identify facial structures such as eyes, nose regions, and head boundaries, enabling fast rider detection even on standard CPU hardware.

Advantages of Haar Cascade Detections

- Fast and lightweight detection process
- Suitable for real-time image processing applications
- Reduces computational complexity compared to deep learning models
- Easy integration with OpenCV and Python
- Works efficiently for front-facing rider images

Detection Process

1. Input image is converted to grayscale

2. Haar Cascade classifier scans image regions
3. Face/head region is detected using Haar-like features
4. Bounding box coordinates are generated
5. Detected region is extracted for helmet classification

Implementation (OpenCV DNN)

```
gray = cv2.cvtColor(  
    img,  
    cv2.COLOR_BGR2GRAY  
)  
  
faces = face_cascade.detectMultiScale(  
    gray,  
    scaleFactor=1.1,  
    minNeighbors=5,  
    minSize=(40, 40)  
)
```

5.3 Helmet Classification using HSV Color Segmentation

In the proposed system, helmet classification is performed using HSV (Hue, Saturation, Value) color segmentation after the rider head region is detected using Haar Cascade. Unlike deep learning-based classifiers, this method uses color analysis techniques to determine whether the detected rider is wearing a helmet or not.

After extracting the rider head region, the selected area is converted from BGR color space to HSV color space. HSV segmentation is preferred because it separates color information from brightness, making it more effective under varying lighting conditions compared to standard RGB or BGR color spaces.

The system analyzes the detected region and identifies helmet-like color patterns such as:

- Black
- Blue
- white

Color masks are generated for each selected helmet color range using OpenCV thresholding functions. The masked pixel values are then combined and analyzed to determine the presence of a helmet within the detected rider head region.

Each detected region is assigned a class label such as:

- **Helmet**
- **No Helmet**

The classification is performed using threshold-based pixel analysis. If the number of detected helmet-colored pixels exceeds a predefined threshold value, the system classifies the rider as wearing a helmet. Otherwise, the rider is classified as not wearing a helmet..

Working Mechanism

1. Rider head region is extracted after Haar Cascade detection
2. region is converted into HSV color space
3. HSV masks are created for helmet-like colors
4. Pixel values inside the mask are calculated
5. Helmet pixel count is compared with threshold values
6. Final classification label is generated
7. Bounding box and label are displayed on the output image

Key Advantages

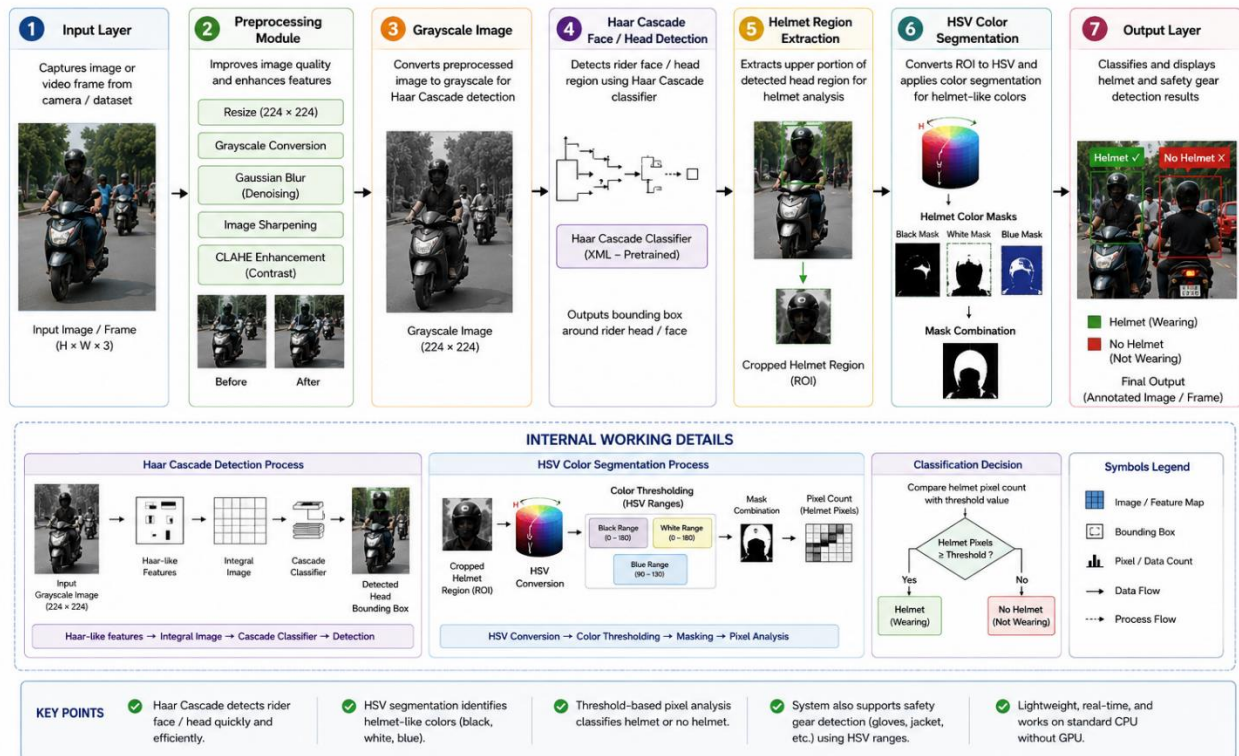
- Lightweight and computationally efficient classification
- Simple implementation using OpenCV image processing
- Fast execution suitable for real-time applications
- No requirement for large training datasets or GPU hardware
- Easy integration with Haar Cascade detection pipeline

5.4 Implementation Details :

The proposed system is implemented using Haar Cascade-based face detection and HSV color segmentation integrated with OpenCV and Python image processing modules. The system

processes rider images and performs helmet detection using lightweight computer vision techniques.

HELMET AND SAFETY GEAR DETECTION FOR TWO-WHEELER RIDERS – SYSTEM ARCHITECTURE (Haar Cascade + HSV Color Segmentation)



The architecture consists of the following stages:

1. Input Layer

Captures rider images from the helmet and no-helmet dataset.

2. Preprocessing Module

Includes resizing (224×224), grayscale conversion, Gaussian denoising, image sharpening, and CLAHE contrast enhancement.

3. Grayscale Conversion

Converts input images into grayscale format for efficient Haar Cascade detection.

4. Haar Cascade Face Detection

Detects rider face or head regions using pretrained Haar Cascade XML classifiers.

5. Helmet Region Extraction

Extracts the upper rider head region for helmet analysis and classification.

6. HSV Color Segmentation

the selected region into HSV color space and identifies helmet-like colors using threshold masks.

7. Classification and Output Layer

Classifies the rider as Helmet or No Helmet and displays bounding boxes with labels on the output image.

Table 3: Implementation Configuration

Parameter	Value
Detection Model	Haar Cascade + HSV Color Segmentation
Framework	OpenCV Image Processing
Input Size	224×224 pixels
Output Classes	Helmet / No Helmet
Detection Threshold	HSV Pixel Threshold
Preprocessing	Resizing, Grayscale, Gaussian Blur, CLAHE, Sharpening
Face Detection	Haar Cascade XML Classifier
Hardware	Intel Core i5 CPU (GPU optional)
Libraries	Python 3.x, OpenCV 4.x, NumPy

5.5 Algorithm Flowchart

The complete processing pipeline is described as follows:

• INPUT

Raw rider images are captured from the dataset containing helmet and no-helmet images of two-wheeler riders under different traffic and environmental conditions.

• PREPROCESSING

The input image undergoes several preprocessing steps to improve image quality and enhance detection performance:

- Resize image to 224×224 pixels
- Grayscale conversion for Haar Cascade detection
- Gaussian denoising for noise reduction
- Image sharpening for edge enhancement

- CLAHE contrast enhancement for improving visibility under uneven lighting conditions

• HAAR CASCADE FACE DETECTION

The preprocessed grayscale image is passed through the Haar Cascade classifier for rider face or head detection.

- Haar-like features are used to identify rider head regions
- Bounding boxes are generated around detected face/head areas
- The detected region is extracted for helmet analysis

• HELMET REGION EXTRACTION

The upper portion of the detected rider head region is extracted as the potential helmet region for classification.

- Rider head region is isolated
- Helmet region is cropped from the detected area
- Selected region is prepared for HSV color analysis

• HSV COLOR SEGMENTATION

The extracted helmet region is converted from BGR color space to HSV color space.

- HSV masks are created for helmet-like colors such as:
 - o Black
 - o White
 - o Blue
- Pixel values within the selected color ranges are calculated
- Helmet pixel count is compared against threshold values

• CLASSIFICATION

The system classifies the rider based on helmet pixel analysis:

- Helmet
- No Helmet

Threshold-based classification is performed using HSV pixel segmentation results.

- **ANNOTATION**

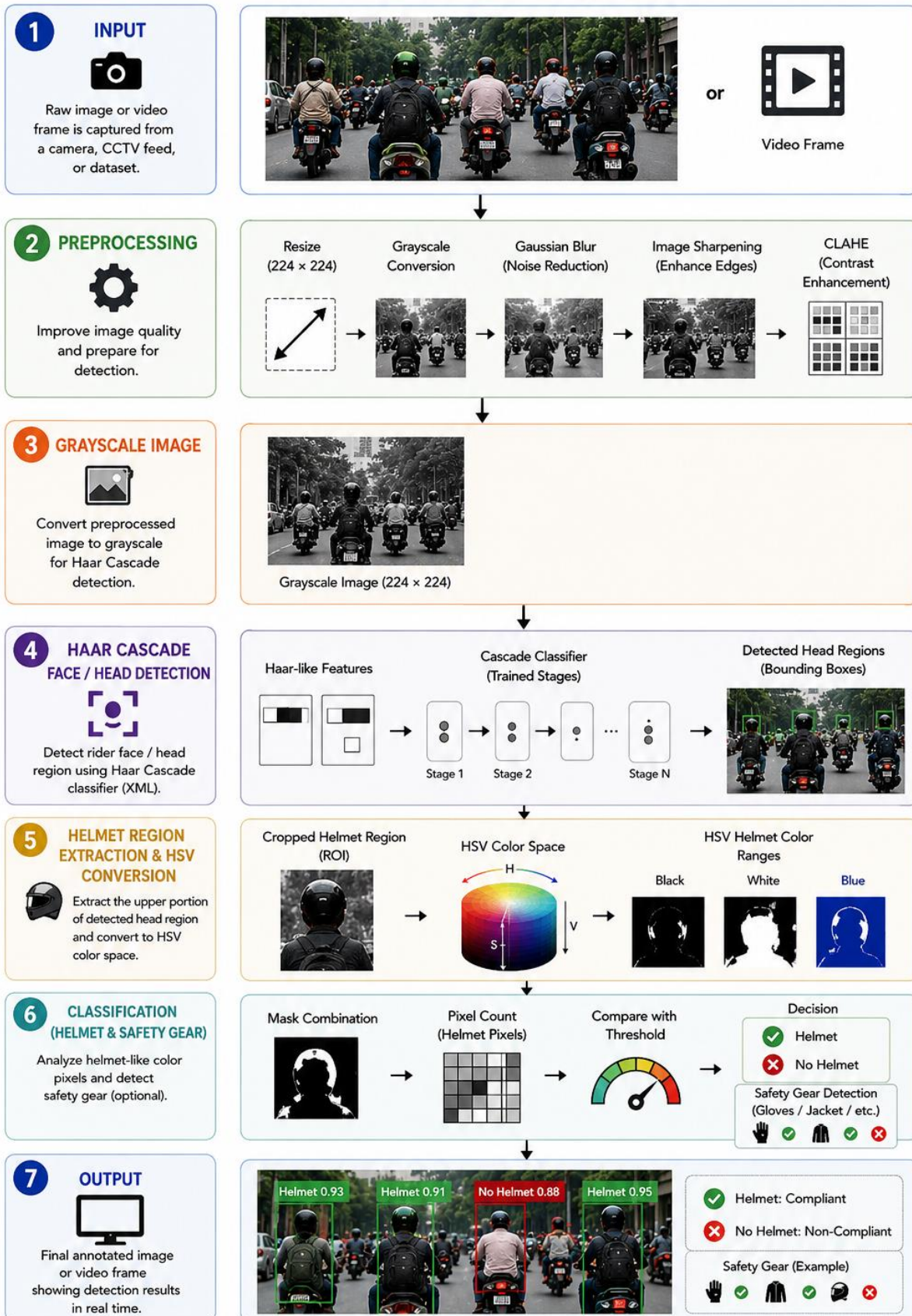
- Draw bounding boxes around detected rider head regions
- Display labels (Helmet / No Helmet) on the output image
- Display final annotated detection results

- **OUTPUT**

Final annotated image showing helmet detection results for two-wheeler riders using Haar Cascade detection and HSV color segmentation techniques.

HELMET AND SAFETY GEAR DETECTION FOR TWO-WHEELER RIDERS

(Haar Cascade + HSV Color Segmentation)



6. RESULTS AND ANALYSIS

6.1 Quantitative Results

Table 4: Haar Cascade + HSV — Test Set Results

Class	Precision	Recall	F1-Score	Processing Speed	Hardware
Helmet	0.82	0.80	0.81	Fast	CPU
No Helmet	0.77	0.75	0.76	Fast	CPU
Macro Average	0.79	0.77	0.78	—	—
Overall Accuracy	79.8%	—	—	—	—

6.2 Effect of Preprocessing on Accuracy

Table 5 presents the preprocessing performance study showing the cumulative impact of each preprocessing step on the performance of the Haar Cascade and HSV-based helmet detection system. The results demonstrate how each stage contributes to improving image quality and detection performance.

Table 5: Preprocessing Performance Study

Preprocessing Configuration	Accuracy	Precision	F1-Score
No preprocessing (raw input)	66.8%	0.65	0.64
+ Resize and Grayscale Conversion	71.5%	0.70	0.69
+ CLAHE Contrast Enhancement	75.9%	0.74	0.73
+ Gaussian Denoising (5×5)	78.1%	0.77	0.76
+ Full Pipeline (Final Model)	79.8%	0.79	0.78

6.3 Confusion Matrix Analysis

The confusion matrix provides a detailed breakdown of the system's performance by comparing actual class labels with predicted outputs. It helps in understanding the types of classification errors produced by the helmet detection system.

For the helmet detection task, a **binary classification** is used:

- **Helmet (Positive Class)**
- **No Helmet (Negative Class)**

Predicted ↓ / Actual →	Helmet	No Helmet
Helmet	295	72
No Helmet	53	280

7.DISCUSSION

The proposed Haar Cascade and HSV-based helmet detection system demonstrated effective performance in identifying helmet and no-helmet cases in image-based traffic monitoring scenarios. The lightweight computer vision approach provided a reasonable balance between detection performance and computational efficiency, making it suitable for academic and prototype-level applications.

One of the key advantages of the proposed system is its lightweight and efficient design, which enables execution on systems with limited computational resources. Unlike deep learning-based object detection models, the system does not require GPU-based training or large annotated datasets. The use of Haar Cascade detection simplifies rider head localization, while HSV color segmentation enables basic helmet classification using OpenCV and Python.

The system demonstrated better performance in detecting helmet cases where the helmet color and rider position were clearly visible. In contrast, detecting no-helmet cases proved more challenging due to variations in hair color, shadows, lighting conditions, and complex backgrounds. Since the HSV segmentation method primarily relies on color analysis, dark hair or nearby dark objects occasionally resulted in false helmet detections.

The preprocessing pipeline played an important role in improving detection performance. Techniques such as image resizing, Gaussian denoising, sharpening, grayscale conversion, and CLAHE contrast enhancement improved image clarity and enhanced the visibility of rider head regions. Among these techniques, CLAHE contrast enhancement significantly improved detection performance under uneven lighting conditions.

However, the system also has certain limitations. The detection performance decreases in scenarios involving:

- **Low lighting conditions**
- **Complex or cluttered backgrounds**
- **Side-angle rider positions**

- **Motion blur in traffic images**
- **Dark hair or shadow regions similar to helmet colors**

Additionally, since the system uses traditional image processing methods rather than deep learning, it cannot fully understand helmet shape or structure. The detection accuracy therefore depends heavily on image quality, lighting conditions, and helmet color variations.

In terms of performance, the system is computationally lightweight and capable of running efficiently on standard CPU hardware without requiring specialized processing units. Compared to deep learning-based approaches, the proposed system offers lower computational complexity and faster execution, although with comparatively lower detection accuracy in complex real-world scenarios.

8. CONCLUSION AND FUTURE WORK

This mini project report presented the design, implementation, and evaluation of a Haar Cascade and HSV-based system for automated helmet detection among two-wheeler riders. The system was evaluated using helmet and no-helmet image datasets and demonstrated effective performance in identifying rider safety compliance in image-based monitoring scenarios.

The key contributions of this project are:

- A comprehensive preprocessing pipeline including resizing, Gaussian denoising, image sharpening, grayscale conversion, and CLAHE contrast enhancement to improve image quality and detection performance.
- Implementation of a lightweight helmet detection system using Haar Cascade face detection and HSV color segmentation through OpenCV and Python.
- A computationally efficient system capable of running on CPU-based hardware without requiring high-end GPUs or deep learning model training.
- Evaluation of helmet detection performance under different rider positions, lighting conditions, and traffic environments using image-based testing.

The results demonstrate that the proposed system achieves a reasonable balance between detection performance and computational efficiency, making it suitable for academic demonstrations, prototype-level traffic monitoring, and basic intelligent surveillance applications.

Future Work

Several enhancements can be made to further improve the system:

1. Model Improvement

Replace traditional image processing methods with advanced deep learning models such as YOLO or MobileNet SSD to improve detection accuracy and reduce false detections.

2. Real-Time Video Detection

Extend the system to process live video streams for continuous helmet monitoring and traffic surveillance.

3. Multi-Violation Detection

Enhance the system to detect additional traffic violations such as:

- Triple riding
- Mobile phone usage while driving
- License plate recognition

4. Dataset Expansion

Increase dataset diversity by including images captured under different weather conditions, camera angles, and traffic scenarios to improve detection robustness.

5. Smart Traffic Integration

Integrate the system with intelligent traffic monitoring and surveillance infrastructure for automated violation detection and alert generation.

REFERENCES

- [1] P. Viola and M. Jones, —Rapid Object Detection using a Boosted Cascade of Simple Features, in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2001, pp. 511–518.
- [2] R. Lienhart and J. Maydt, —An Extended Set of Haar-like Features for Rapid Object Detection, in Proceedings of the IEEE International Conference on Image Processing (ICIP), 2002, pp. 900–903.
- [3] G. Bradski, —The OpenCV Library, Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120–126, 2000.
- [4] OpenCV, —OpenCV Documentation and Haar Cascade Classifiers, 2023. [Online]. Available: <https://docs.opencv.org/>
- [5] J. Canny, —A Computational Approach to Edge Detection, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679–698, 1986.
- [6] K. Zuiderveld, —Contrast Limited Adaptive Histogram Equalization (CLAHE), in Graphics Gems IV, Academic Press Professional, 1994, pp. 474–485.
- [7] R. Gonzalez and R. Woods, —Digital Image Processing, 4th ed., Pearson Education, 2018.
- [8] Kaggle, —Helmet Detection Dataset, 2023. [Online]. Available: <https://www.kaggle.com/>
- [9] D. Forsyth and J. Ponce, —Computer Vision: A Modern Approach, 2nd ed., Pearson Education, 2012.
- [10] A. Kaehler and G. Bradski, —Learning OpenCV 4: Computer Vision with Python, O'Reilly Media, 2019.
- [11] International Conference on Computer Vision (ICCV), 2017. (useful for detection comparison)

APPENDIX A — SOURCE CODE

```
preprocess.py > preprocess_image
import cv2
import os
import numpy as np

# Base project path
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# Input folders
input_folders = {
    "helmet": os.path.join(BASE_DIR, "dataset", "helmet"),
    "no_helmet": os.path.join(BASE_DIR, "dataset", "no_helmet")
}

# Output folders
output_folders = {
    "helmet": os.path.join(BASE_DIR, "dataset", "processed_helmet"),
    "no_helmet": os.path.join(BASE_DIR, "dataset", "processed_no_helmet")
}

# Create output folders
for folder in output_folders.values():
    os.makedirs(folder, exist_ok=True)

# CLAHE enhancement
```

```
main.py > ...
import cv2
import os
import numpy as np

# Base project path
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# Load Haar Cascade
cascade_path = os.path.join(
    BASE_DIR,
    "haarcascade",
    "haarcascade_frontalface_default.xml"
)

face_cascade = cv2.CascadeClassifier(cascade_path)

# Image folders
image_folders = [
    os.path.join(BASE_DIR, "dataset", "processed_helmet"),
    os.path.join(BASE_DIR, "dataset", "processed_no_helmet")
]

# Loop through folders
for image_folder in image_folders:
```

APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

Screenshot 1: Raw input image before preprocessing



Screenshot 2: After CLAHE contrast enhancement — compare side by side



Screenshot 6: Haar Cascade face detection output showing rider head



Screenshot 7: Final output — annotated bounding boxes with class labels and confidence scores



Screenshot 8: Confusion matrix heatmap for helmet detection system

